ZentroMart Voice Support Agent

VoiceOps 2026 capstone · From Tokens to Transformers (GenAI x LLM), C&A Club IIT Guwahati x Gnani.ai
Aarna Rathod · BTech CSE, Manipal University Jaipur · 23 October 2026
Code: github.com/aarnarathod114-lab/zentromart-agent

Summary

I built a customer-support agent for ZentroMart, the course's fictional Indian online store. It answers in English, Hindi and Hinglish, grounds its answers in the company knowledge base and cites section IDs, looks up orders only after identity verification, hands over to a human team when policy requires it, and works by voice: the customer's speech is streamed to Gnani STT over WebSocket and the reply is spoken back with Gnani TTS over WebSocket. Each step was measured with one shared scorer on the course's practice data.

Step	What it adds	Headline result
1 Baseline	LLM only, no documents	30.1% of required facts; 0% evidence
2 RAG	Hybrid multilingual retrieval, business-day calendar	86.5% facts, 86.5% decisions, 76.2% evidence
3 Conversations	Memory, identity-checked tools, tickets	92.7% decisions, 98.1% tool calls, 0 forbidden claims
4 Voice	Gnani STT/TTS, live streaming	Transcript 2.0 s after the customer stops; ASR text costs ~12 points

1. Problem and data

The dataset is synthetic and frozen on **20 October 2026**, which is used as "today" for every date. It has 21 knowledge-base documents (policies, agent guides, help articles, FAQs including one in Hindi, and a marketing page), seven data tables (1,000 orders, 70 products, pincodes, holidays), two mock tools, 52 practice questions and 23 practice conversations (96 customer turns). The README lists what makes it hard, and each item shaped a design decision:

- **Documents disagree**: a precedence rule (KB-00.1) says current policy beats agent guide, help, FAQ and marketing.
- **Policies change**: every document has a *status* and *effective_from*; e.g. electronics delivered before 1 Aug 2026 keep a 10-day return window.
- **Customers don't use the documents' words** ("nobody came to collect it" vs "reverse pickup"), and write in Hindi or Hinglish while documents are English.
- **Timelines use business days** (Mon–Sat, skipping listed warehouse holidays).
- **Verify before you share**: order details only after the last 4 phone digits or delivery pincode match, with at most one retry.

2. Architecture

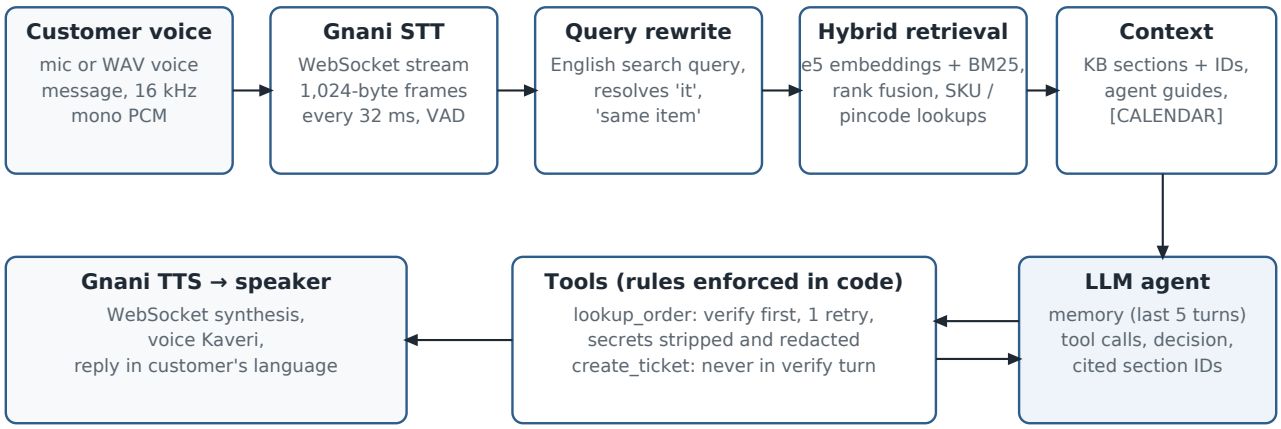


Figure 1. One customer turn. Text input enters at the query-rewrite stage; voice adds Gnani STT before it and Gnani TTS after the agent.

The same agent class serves the text evaluation (Step 3) and the voice front end (Step 4). The scorer `score.py` is shared by every step, so all numbers are directly comparable: decision against the acceptable decisions, required facts (where " / " separates alternatives), forbidden claims, and evidence recall against the gold section IDs. Hindi digits are normalised before matching.

3. Step 1: baseline without documents

The model (gpt-oss-120b on Groq) answered all 52 questions with no documents. It picks a reasonable decision 76.9% of the time, but only 30.1% of required facts appear, because return windows, refund days and fees are ZentroMart-specific and the model guesses them. 11 of its 12 decision errors were "clarify" where an answer was expected: without documents it asks the customer instead. Evidence recall is 0% by construction.

4. Step 2: retrieval-augmented answers

Retriever design

- **Chunking by section ID**: one chunk per `## [ID] Heading`, keeping title, doc_type, status and effective_from; each product row is also a chunk. 154 chunks in total.
- **Hybrid search**: multilingual-e5-small embeddings (paraphrase and Hindi) plus BM25 (exact SKUs and terms), merged by reciprocal rank fusion, with a mild precedence prior (policy > agent guide > help > FAQ > marketing).
- **English query rewrite** by a small model, so Hindi questions reach English policies.
- **Exact lookups** for any SKU or 6-digit pincode in the question; catalog rows get separate slots so they cannot crowd out policies.
- **[CALENDAR] block**: business-day and calendar-day results computed in Python from the dates in the question, written one per line.

Problems found and fixed with small targeted tests

Observation	Cause	Fix
Hindi refund question found no refund policy	Policies are English; Hindi query embeds near the Hindi FAQ	Search with original + English rewrite
SKU question lost the warranty policy	Six catalog rows filled all slots	Separate slots: 6 documents, max 2 catalog rows
Strong policy boost dropped the reverse-pickup help article	Prior of 1.20 outweighed relevance	Gentle prior (1.06 to 0.85)
Both 20B and 120B said 26 Oct instead of 27 Oct	"N business days from a date" read as counting the start day	Calendar plus an explicit counting rule
Invented a "courier slots" rule for midnight delivery	General knowledge filled a gap	Rule: if not described in context, escalate

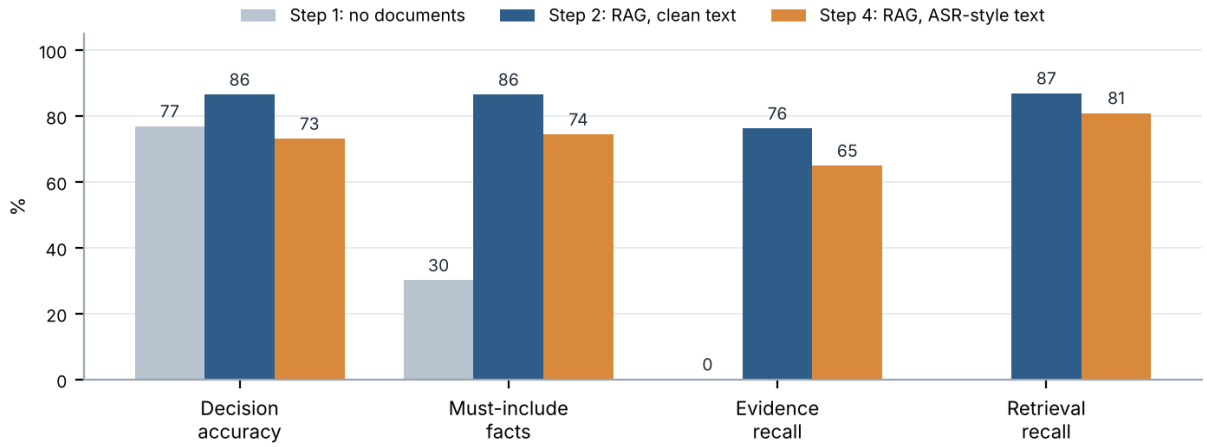


Figure 2. Same 52 questions and model (gpt-oss-120b). Step 1 has no retrieval, so its evidence and retrieval recall are 0.

Metric (52 questions, gpt-oss-120b)	Step 1 baseline	Step 2 RAG
Decision accuracy	76.9%	**86.5%**
Must-include facts (average)	30.1%	**86.5%**
All required facts present	25.0%	**84.6%**
Forbidden-claim hits	0	0
Evidence recall (cited IDs)	0%	**76.2%**
Retrieval recall (gold IDs retrieved)	–	**86.9%**

By reply language, facts reached 85% in English, 100% in Hindi and 82% in Hinglish, so the multilingual retrieval worked. The remaining decision errors are mostly "clarify" where the dataset expects a direct answer.

5. Step 3: multi-turn conversations with tools

The agent keeps the last five exchanges as memory and a conversation state built from tool results. Each turn it rewrites the latest message using recent turns (so "same item" resolves), retrieves policy, always includes the three agent guides and the precedence and business-day rules, and may call `lookup_order` and `create_ticket`. Dates from the verified order (delivery, pickup, refund initiation) are fed into the calendar, which is how it gives the exact return deadline of 27 October 2026 for sneakers delivered on 12 October.

Safety rules enforced in code, not only in the prompt

- The verified order record is passed to the model **without** the delivery pincode or customer ID, so it cannot leak them; verification digits are replaced by **** in replies and ticket summaries.
- At most one retry: after two failed verifications the tool refuses further attempts.
- The decision label follows what the tools actually did: a created ticket means "escalate"; an unverified lookup means "clarify".
- A ticket cannot be created in the same turn as verification. The small model kept escalating as soon as it saw a damage report in the record, before the customer had asked.
- If the model replies in plain text instead of JSON (Gemini Flash-Lite did), a second JSON-mode call labels the decision and cited IDs.

Metric (23 conversations, 96 turns, gemini-flash-lite)	Result
Decision accuracy	**92.7%**
Must-include facts	**88.0%**
Forbidden claims ("approved", leaked secrets)	**0**
Expected tool calls made (54 turns)	**98.1%**
Evidence recall	70.5%
Turns fully correct	82.3%
Unexpected tickets	3 (all early escalations; then blocked in code)

Scores were similar across English, Hindi and Hinglish (81–83% fully correct turns). The remaining errors are early escalations in repeat-contact cases and labels in failed-verification flows.

6. Step 4: voice with Gnani

`voice.py` uses the official gnani-vachana SDK. STT runs over WebSocket with raw 16-bit PCM at 16 kHz mono in 1,024-byte frames, one every 32 ms, as the protocol requires; Gnani's voice-activity detection closes each segment and returns a transcript event. TTS runs over WebSocket (voice Kaveri). `run.py` supports a live microphone, voice-message WAV files, or typed input, saves audio locally and logs per-turn timings.

Making it real-time

The first version recorded seven seconds and only then streamed the clip, so the transcript arrived about 16 seconds after the customer finished. The final version streams microphone frames to Gnani *while the customer speaks* and only waits for the last segment afterwards.

Stage	First version	Final version
STT: transcript after the customer stops	15.6–16.9 s	**2.0 s**
Agent (Groq gpt-oss-20b)	2.7–9.6 s	2.9 s unthrottled; up to 40 s when the free-tier per-minute limit was hit
TTS synthesis (Gnani WebSocket)	2.0–3.6 s	2.3–2.6 s

Two live sessions

- **Success path**: greeting → order ID (Gnani produced the digits correctly) → last four phone digits → verified → "my TV was damaged, approve a replacement" → ticket created for the human team. The agent never said "approved", and with the code rule it did not escalate in the verification turn.
- **Real speech-recognition error**: the order number "8978 2837" was transcribed with the first 8 heard as the Hindi letter "e", giving a wrong 7-digit ID. The correct phone digits therefore failed, the agent allowed exactly one retry, then shared nothing and offered a

human. Replies were in Hindi after adding an explicit language note for Devanagari transcripts.

Robustness to speech-recognition errors

Each practice question also has an ASR-style version (e.g. "ELE-0012" becomes "ele-zero zero one to"). Running the unchanged Step 2 pipeline on these, with the same model:

Metric (52 questions, gpt-oss-120b)	Clean text	ASR-style text
Decision accuracy	86.5%	73.1%
Must-include facts	86.5%	74.4%
Retrieval recall	86.9%	80.9%
Evidence recall	76.2%	64.9%
Forbidden-claim hits	0	**0**

The drop comes mainly from spoken identifiers: SKU questions that were fully correct on clean text failed because the exact lookup no longer matched, and garbled questions led the agent to ask the customer to clarify. It never invented a policy. The live session showed the same weakness on order numbers.

7. Working within free tiers

Only free, no-card services were used. This drove several engineering choices:

- **Groq** free tier gives every model 8K tokens/minute and 200K tokens/day. Step 2's full run used ~139K tokens, so Step 1, 2 and the ASR evaluation each fit in one day on gpt-oss-120b.
- Step 3 needs ~600K tokens, so it ran on **Gemini Flash-Lite** (free tier). Gemini Flash took ~250 s per request on a 5K-token prompt and was unusable; Flash-Lite took ~12 s.
- Voice uses Groq for its low per-request latency; the agent code switches provider by model name.
- Evaluations are resumable and stop cleanly at a daily limit, saving finished conversations.

8. Limitations and next steps

- **Spoken numbers**: add an ASR normaliser that turns spoken or mis-heard digits back into IDs ("ele-zero zero one to" → ELE-0012) and asks the customer to confirm an order number before lookup. This targets the main source of the Step 4 drop.
- **Failed verification**: after the second failure the agent offered a human but did not create a ticket, and the turn was labelled "clarify". The code should create the handover ticket and label it "escalate".
- **Over-clarifying**: several single questions were answered with "clarify" where the dataset expected an answer.
- **Latency** numbers include free-tier throttling; Step 3 averaged 75 s per turn on Gemini, almost all queueing.
- Step 3 was scored on a smaller model than Step 2 because of daily token limits, so the two are not a like-for-like model comparison.
- Gnani's hi-IN STT writes Hinglish speech in Devanagari, so voice replies are in Hindi script. Voice recordings are not committed, for privacy.

9. Reproducing the results

Python 3.12, `pip install -r requirements.txt`, the course dataset in `dataset/`, and a `.env` with GROQ_API_KEY, GEMINI_API_KEY and GNANI_API_KEY. Then:

```
python step1_baseline.py openai/gpt-oss-120b
python step2_rag.py openai/gpt-oss-120b
python step3_conversations.py gemini-flash-lite-latest
python step4_asr_eval.py openai/gpt-oss-120b
python run.py --model openai/gpt-oss-20b (live voice)
```

Every run writes a per-item CSV to `results/`; `inspect_results.py` prints the details of any question.